

ELEC-H550: Embedded Systems Security

Jan Tobias Mühlberg

jan.tobias.muehlberg@ulb.be

Université libre de Bruxelles, Cybersecurity Research Center, Belgium

ULB, Brussels, 2026/2027

Session 3: Security in the Software Development Life-Cycle



OPEN SOURCE COMMUNITY

By [Colin Neagle](#), Assistant Community Editor, Network World | AUG 26, 2015 10:02 AM PT

About

The Open Source subnet's hub.

Smart refrigerator hack exposes Gmail login credentials

A bonus feature on a smart home product becomes a security liability.



RELATED



Welcome to the smart home
... of horror!

Source: <https://www.networkworld.com/article/2976270/internet-of-things/smart-refrigerator-hack-exposes-gmail-login-credentials.html>

Security in the Software Development Life-Cycle

The Joy of Tech™ by Nitrozac & Snaggy

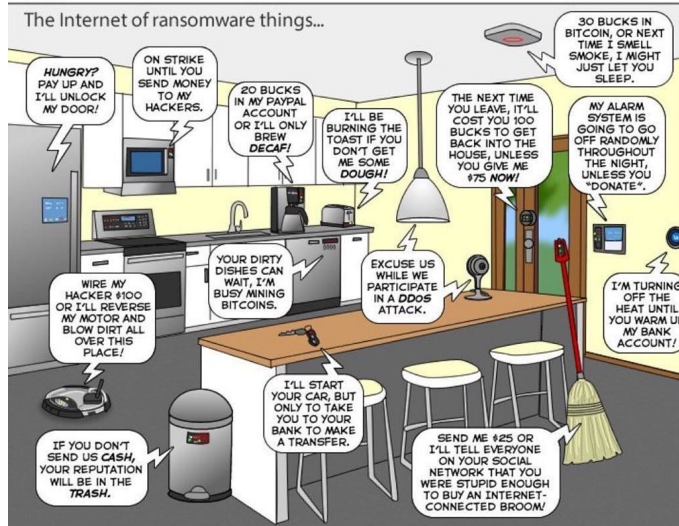
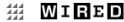


Image source: <http://www.joyoftech.com/joyoftech/joyarchives/2340.html>

Security in the Software Development Life-Cycle



The Jeep Hackers Are Back to Prove Car Hacking Can Get Much

SHARE



SHARE
2841



TWEET



COMMENT



EMAIL

ANDY GREENBERG SECURITY 08.01.16 03:30 PM

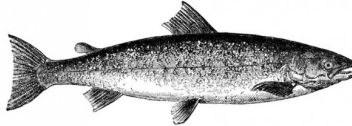
THE JEEP HACKERS ARE BACK TO PROVE CAR HACKING CAN GET MUCH WORSE



Source: <https://www.wired.com/2016/08/jeep-hackers-return-high-speed-steering-acceleration-hacks/>

Security in the Software Development Life-Cycle

Security by optimism and prayer



Expert

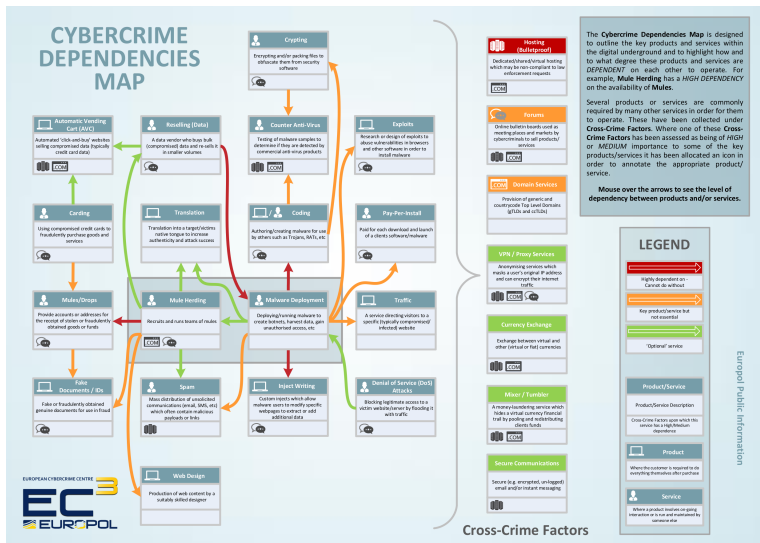
Hoping Nobody
Hacks You

ORLY?

@ThePracticalDev

Image source: <https://twitter.com/thepracticaldev/status/764829038997278720>

Security in the Software Development Life-Cycle



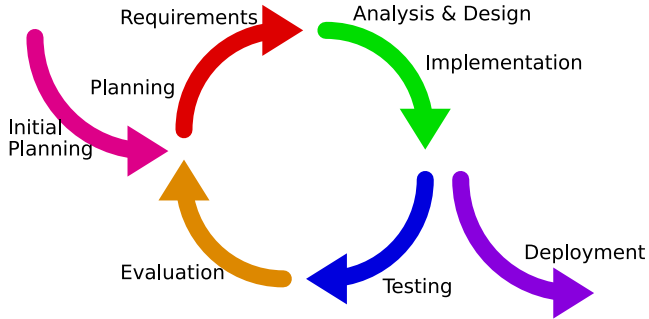
Source: <https://www.europol.europa.eu/publications-documents/cybercrime-dependencies-map>

Security in the Software Development Life-Cycle



Source: <https://www.xkcd.com/1938/>

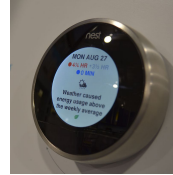
Security in the Software Development Life-Cycle



Vulnerabilities can be introduced in any phase. Key is early detection and remediation, which should be based on rigorous assessments.

Common approach: Deploy, pray, get breached and patch.

Security in the Software Development Life-Cycle

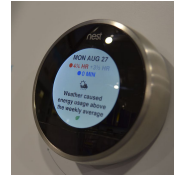


Source of images 1, 2, 3: <https://en.wikipedia.org/>

Security in the Software Development Life-Cycle

1 Understand the system.

- Context, hardware, software, data, users, use cases, etc.



Source of images 1, 2, 3: <https://en.wikipedia.org/>

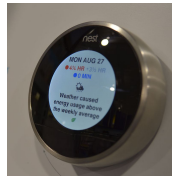
Security in the Software Development Life-Cycle

1 Understand the system.

- Context, hardware, software, data, users, use cases, etc.

2 Understand the security requirements.

- Requirements are not features!
- "We have 2-factor authentication!"**
- "Only authenticated users can do X. Two-factor authentication is required for all users. All X are logged, detailing time, user and properties of X."**



Source of images 1, 2, 3: <https://en.wikipedia.org/>

Security in the Software Development Life-Cycle

1 Understand the system.

- Context, hardware, software, data, users, use cases, etc.

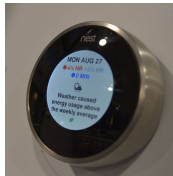
2 Understand the security requirements.

- Requirements are not features!
- "We have 2-factor authentication!"**
- "Only authenticated users can do X. Two-factor authentication is required for all users. All X are logged, detailing time, user and properties of X."**

3 Understand the attacker.

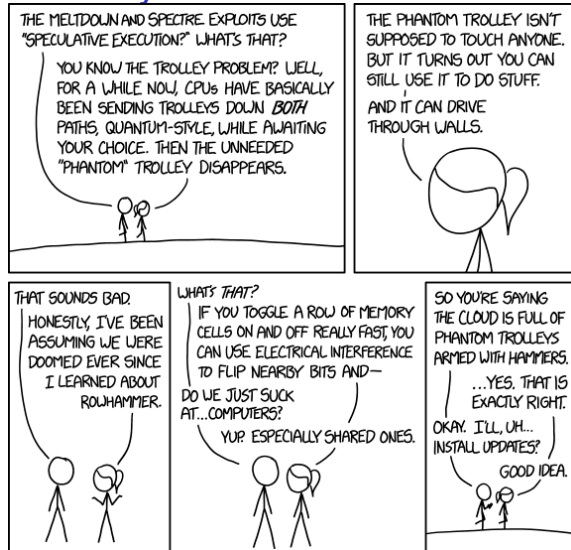
- "Attackers can listen to all communication, can drop, reorder or replay messages, may compromise Y% of the system, can't break crypto." [DY81]

Source of images 1, 2, 3: <https://en.wikipedia.org/>



Security in the Software Development Life-Cycle

“New zero-day vulnerability: In addition to rowhammer, it turns out lots of servers are vulnerable to regular hammers, too.”

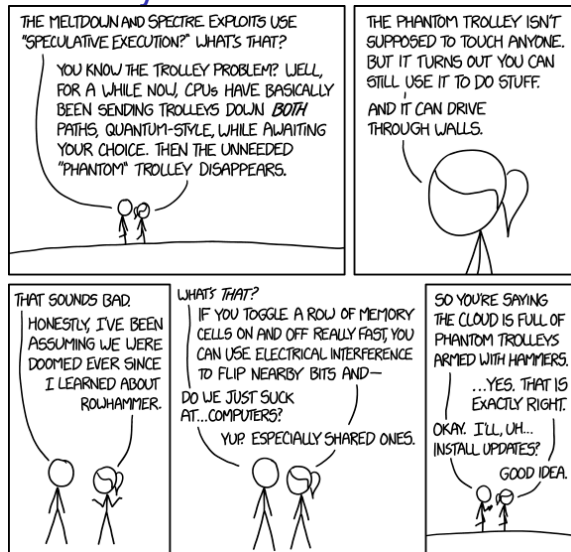


Source: <https://xkcd.com/1938/>

Security in the Software Development Life-Cycle

“New zero-day vulnerability: In addition to rowhammer, it turns out lots of servers are vulnerable to regular hammers, too.”

- 1 Understand the system.
- 2 Understand the security requirements.
- 3 Understand the attacker.

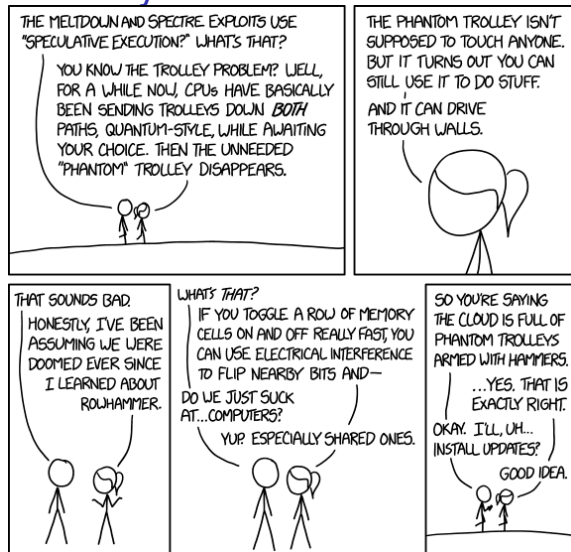


Source: <https://xkcd.com/1938/>

Security in the Software Development Life-Cycle

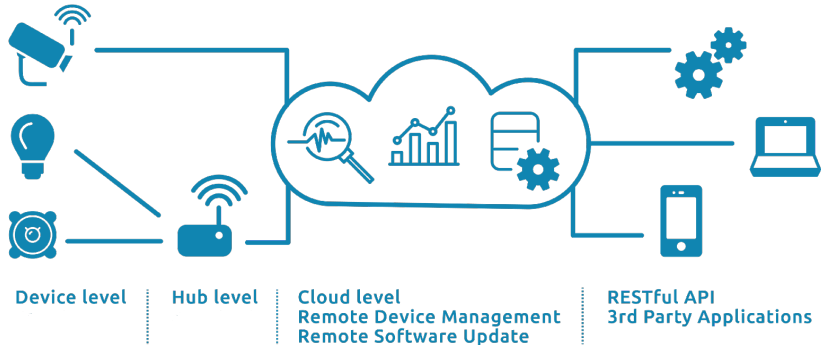
“New zero-day vulnerability: In addition to rowhammer, it turns out lots of servers are vulnerable to regular hammers, too.”

- 1 Understand the system.
- 2 Understand the security requirements.
- 3 Understand the attacker.
- 4 Understand and embrace change!
 - Discovery of vulnerabilities
 - Different understanding of the system
 - New (functional|security) requirements
 - New attacks, different attackers



Source: <https://xkcd.com/1938/>

Security in the Software Development Life-Cycle



Understanding can be really difficult! What stake holders are involved? What are their objectives and abilities? What hardware and software is involved? Software quality? Data flows? Security requirements and guarantees?

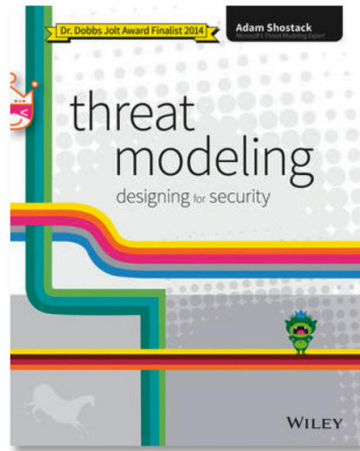
Image source: <https://medium.com/connected-news/iot-foundation-what-is-an-iot-platform-c37c5e72d4a0>

Security in the Software Development Life-Cycle

Read a good book!

“Threat Modeling: Designing for Security”

Adam Shostack, Wiley, 2014 [Sho14]



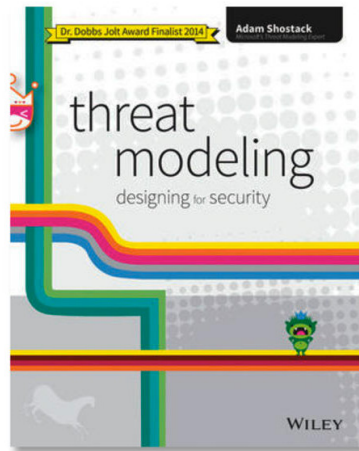
Security in the Software Development Life-Cycle

Read a good book!

“Threat Modeling: Designing for Security”

Adam Shostack, Wiley, 2014 [Sho14]

Do some practical threat modeling!



Security in the Software Development Life-Cycle

Read a good book!

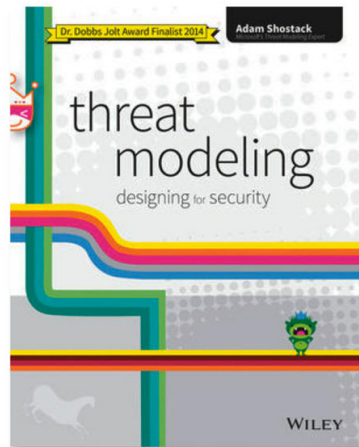
“Threat Modeling: Designing for Security”

Adam Shostack, Wiley, 2014 [Sho14]

Do some practical threat modeling!

Learn about security tech!

This course: Vulnerabilities & defences
in software and hardware



Security in the Software Development Life-Cycle

Read a good book!

“Threat Modeling: Designing for Security”

Adam Shostack, Wiley, 2014 [[Sho14](#)]

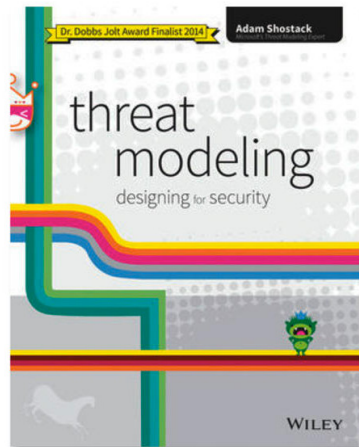
Do some practical threat modeling!

Learn about security tech!

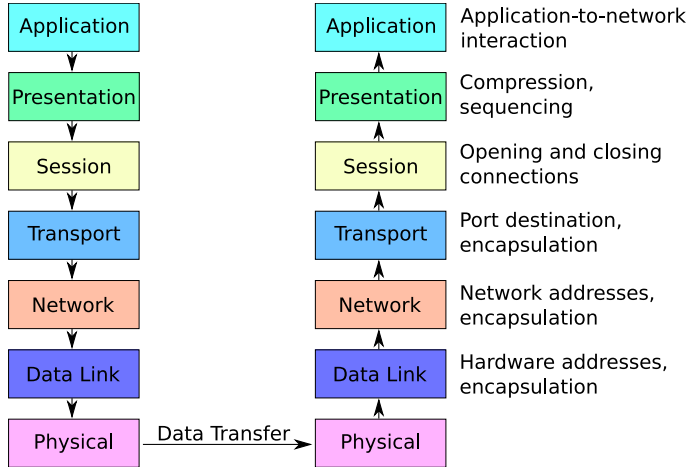
This course: Vulnerabilities & defences
in software and hardware

Be prepared...

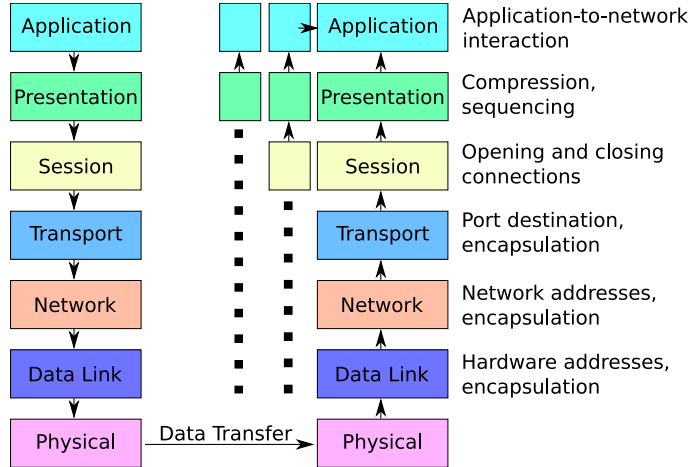
...to reconsider your system design once
some tech is broken or something better
comes along!



Understanding Threats



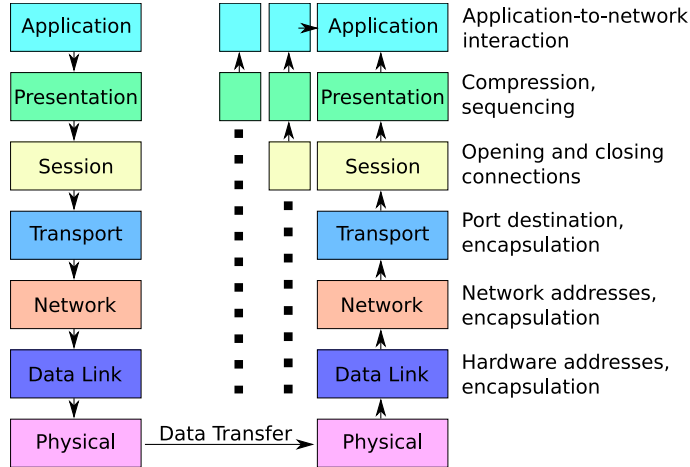
Understanding Threats



Understanding Threats

Questions:

- Where are our assets?
- What are the threats?
- Where are the attackers?
- What do they do?
- Where do we put our defences?

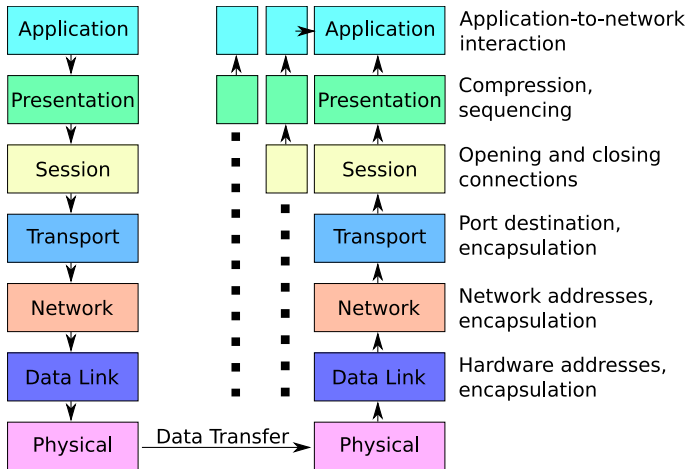


Understanding Threats

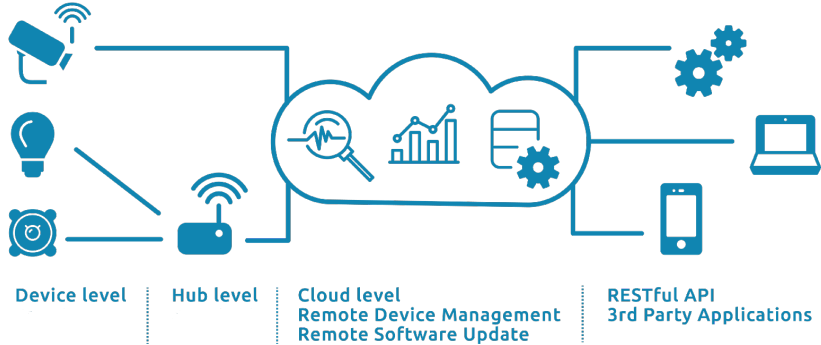
Questions:

- Where are our assets?
- What are the threats?
- Where are the attackers?
- What do they do?
- Where do we put our defences?

It's hard to detect and defend against application-layer attacks at any lower level.



Understanding Threats



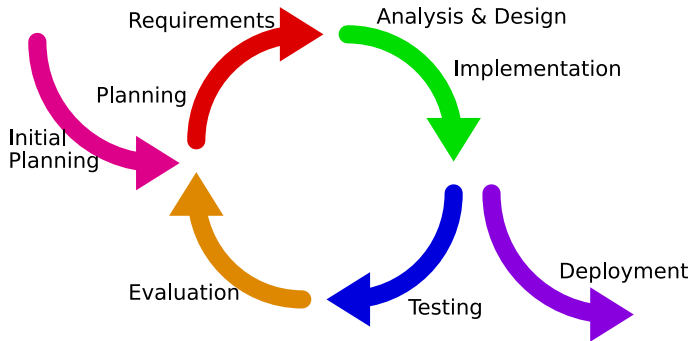
Questions: Where are our assets? What are the threats? Where are the attackers? What do they do? Where do we put our defences?

Image source: <https://medium.com/connected-news/iot-foundation-what-is-an-iot-platform-c37c5e72d4a0>

Understanding Threats

Questions:

- Where are our assets?
- What are the threats?
- Where are the attackers?
- What do they do?
- Where do we put our defences?

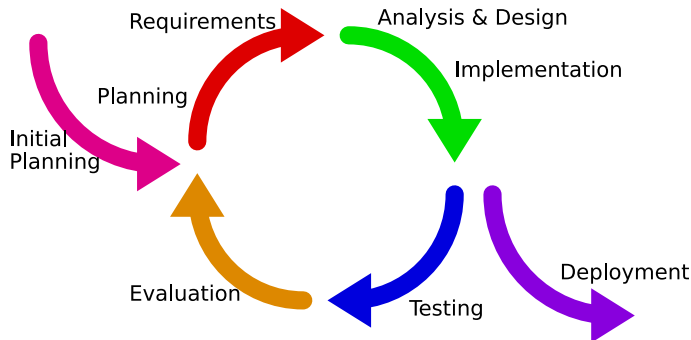


Understanding Threats

Questions:

- Where are our assets?
- What are the threats?
- Where are the attackers?
- What do they do?
- Where do we put our defences?

→ **Threat Modeling**



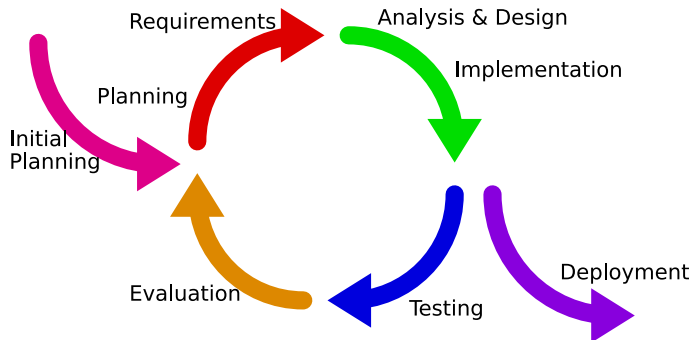
Understanding Threats

Questions:

- Where are our assets?
- What are the threats?
- Where are the attackers?
- What do they do?
- Where do we put our defences?

→ **Threat Modeling**

→ Part of “Analysis & Design”



Understanding Threats

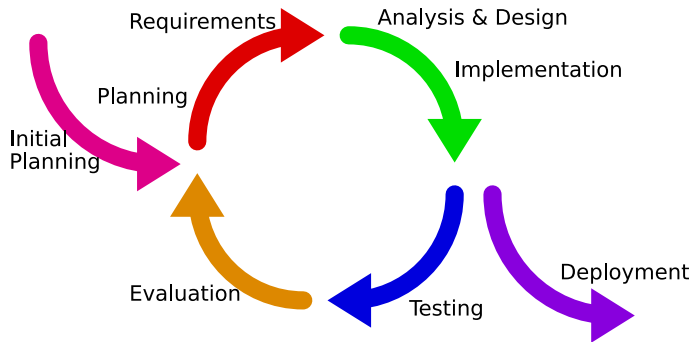
Questions:

- Where are our assets?
- What are the threats?
- Where are the attackers?
- What do they do?
- Where do we put our defences?

→ Threat Modeling

→ Part of “Analysis & Design”

→ Re-do as your requirements and design change!



Understanding Threats: Threat Modeling

What is a good way to do threat modeling?

- No single way of doing threat modeling, just **ways that find good threats**
 - Good threats uncover work that needs to be done to build a more secure system
 - Over time, you will develop your own way of doing threat modeling
- Threat modeling is about **technique vs. repertoire**
 - By practising the technique, your mental set of examples develops (i.e. repertoire)
 - With more experience and repertoire threat modeling will become easier
- In this session, we will go deeper into Microsoft's **STRIDE** technique
 - This will get you started with threat modeling, so you can make it your own
 - In the second part of the session, we will do a hands-on case study

Understanding Threats: Threat Modeling

Why should you do threat modeling?

- Threat modeling helps you **find security problems early in the life-cycle**
 - First threat modeling activities can take place during conception of the system
 - Threat modeling is an iterative process, and will help to keep finding security problems
- Understand and concretise your security **requirements**
 - Threat modeling will raise questions that you should have an answer for
 - Concrete security requirements help you focus your efforts efficiently
- Complement other security techniques to **address more problems**
 - Tools like static analysis will help you discover bugs, such as buffer overflows or SQL injection
 - Threat modeling will help you find design flaws, such as missing data encryption

Understanding Threats: Threat Modeling

System Diagram → Identify Threats → Consider Mitigations → Validate

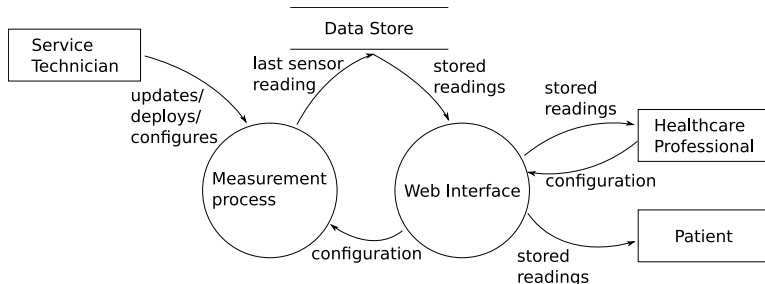
- **1. What are we building? → System Diagram**
- **2. How can attackers harm the system? → Threats**
- **3. How do we defend against threats? → Mitigations**
- **4. Validate, Report, Communicate!**

We commonly use **Data Flow Diagrams** (DFDs) to describe a system; other diagrams such as **Sequence Diagrams** or **Use Case Diagrams** can also work!

Understanding Threats: Threat Modeling

1. System Diagram: What are we building?

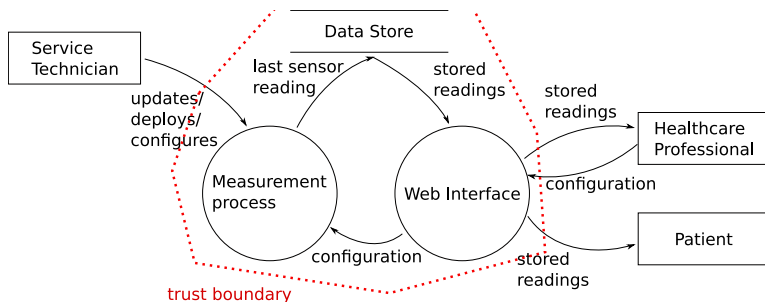
- Scope, context and objectives
- Interactions, actors, processes, data stores
- Attack surface, trust boundaries
- Pick your tools and resources: **Time and people!** What **diagrams** and at which **granularity**? What do you exclude and why?



Understanding Threats: Threat Modeling

1. System Diagram: What are we building?

- Scope, context and objectives
- Interactions, actors, processes, data stores
- Attack surface, trust boundaries
- Pick your tools and resources: **Time and people!** What **diagrams** and at which **granularity**? What do you exclude and why?



Understanding Threats: Threat Modeling

1. System Diagram: What are we building?

- You should be able to explain the use cases with the diagram
 - Without changes or referring to elements not present in the diagram!
- Rules of thumb to evaluate a DFD
 - Add all details needed to explain security-relevant behaviour
 - Avoid *sometimes* and *also* to explain behaviour of an element
 - Processes should not be miracles or black holes, data comes in and goes out
 - But external entities can be!
 - Data sinks are not useful, someone is always using the data
 - Data does not move itself between data stores – Add a process that does this
 - Mechanisms controlling data flows should be shown: firewalls, access control systems, ...

Understanding Threats: Threat Modeling

2. Treads: How can attackers harm the system?

- Everyone can think of fairly intuitive ways to find threats
 - brainstorming, thinking up movie plots, review literature, ...
 - Intuitive approaches are hard to measure, and hard to reproduce consistently
- Threat modeling is about a systematic approach to identify threats
 - The exact approach can vary, but key is that it must be systematic and reproducible
 - E.g. attack libraries, attack trees, STRIDE, ...
- STRIDE is the most common methodology to do threat modeling
 - Enumeration of various kinds of abstract threats that can exist in a system:
Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
 - Fits well onto a DFD diagram, which is why both are generally used together

2. Treads: How can attackers harm the system?

- STRIDE is a mnemonic to refer to a group of threats
 - **Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege**
 - It's not an exhaustive list, neither a taxonomy nor a categorisation
 - The threats in STRIDE are abstract, but become concrete when instantiated
 - Concrete threat catalogues exist for specific domains!
- STRIDE brings structure to threat modeling
 - It helps you go over the list of possible threats
 - You think of threats as you go along
- Threats are closely related to security properties
 - A threat violates a security property
 - But appropriate mitigations counter a threat

2. Treads: How can attackers harm the system?

Table: Selected threats to the 5G Core Network with associated threat severities, likelihoods, and risks. Risk ratings are used to prioritise threats.

Mnemonic: Title	Description, Prerequisites, Severity, Likelihood, & Risk
ElevPriv5G: Unauthorised access to 5G core functions	<i>Description:</i> If an attacker manages to compromise a pod running 5G core services (AMF, SMF, or UPF) they get full access to all user data handled by these functions. Moreover the attacker may obtain credentials allowing lateral movement into the 5G core network or user devices or to send malicious requests to other network functions. <i>Prerequisites:</i> The attacker needs to run code on an edge platform that also hosts 5G core functions and break their network isolation and authentication / authorisation to gain access. <i>Severity: High; Likelihood: Low; Risk: 4</i>
DoS5GRadio: Disrupting the 5G Radio Access Network	<i>Description:</i> Since 5G RAN allows random access, attackers may start DoS attacks against it on different layers of the RAN protocol stack. Such attacks may limit service availability to other users of an edge platform. <i>Prerequisites:</i> The attacker needs to be in the coverage area of radio antennas and base stations servicing the edge platform. <i>Severity: Medium; Likelihood: Medium; Risk: 5</i>

2. Treads: How can attackers harm the system?

- **Spoofing:** Can an attacker gain access using a false identity?
- **Tampering:** Can an attacker modify data as it flows through the application?
- **Repudiation:** If an attacker denies doing something, can we prove he did it?
- **Information Disclosure:** Can an attacker gain access to private or potentially injurious data?
- **Denial of Service:** Can an attacker crash or reduce the availability of the system?
- **Elevation of Privilege:** Can an attacker assume the identity of a privileged user?

Understanding Threats: Threat Modeling

2. Treads: How can attackers harm the system?

Apply the relevant parts of STRIDE to each item on the diagram:

- External Entity – S, R
- Process – S, T, R, I, D, E
- Data store, data flow – T, I, D
- Logging facilities – T, I, D, and R

	S	T	R	I	D	E
External Entity	✓		✓			
Process	✓	✓	✓	✓	✓	✓
Data Store		✓		✓	✓	
Data Flow		✓		✓	✓	
Log Data Store		✓	✓	✓	✓	

Understanding Threats: Threat Modeling

2. Treads: How can attackers harm the system?

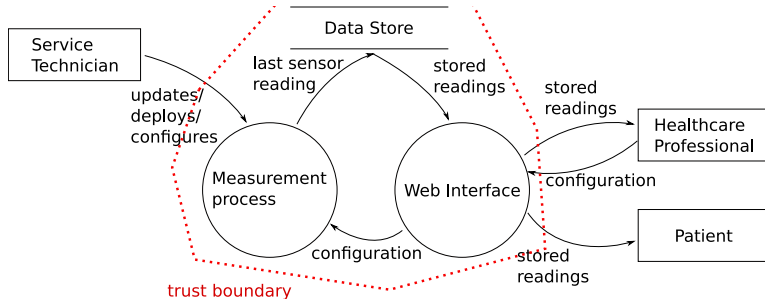
Apply the relevant parts of STRIDE to each item on the diagram:

- External Entity – S, R
- Process – S, T, R, I, D, E
- Data store, data flow – T, I, D
- Logging facilities – T, I, D, and R

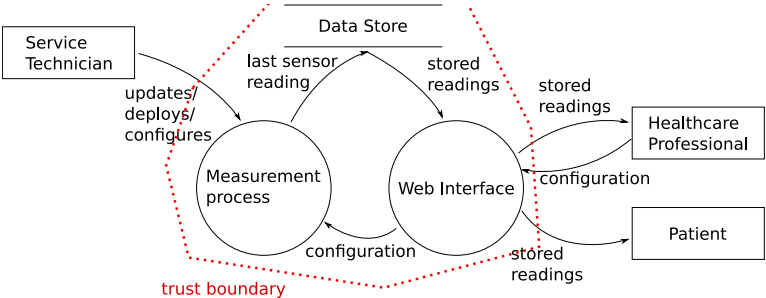
3. Mitigations: How do we defend against threats?

	S	T	R	I	D	E
External Entity	✓		✓			
Process	✓	✓	✓	✓	✓	✓
Data Store		✓		✓	✓	
Data Flow		✓		✓	✓	
Log Data Store		✓	✓	✓	✓	

Understanding Threats: Threat Modeling

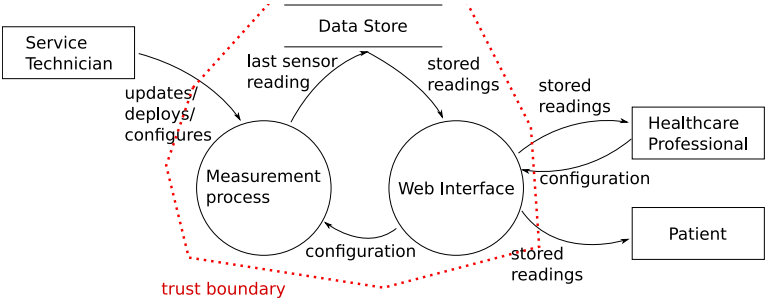


Understanding Threats: Threat Modeling



	Service Technician		-->	Measurement Process	
	Mitigations	Vulnerabilities		Mitigations	Vulnerabilities
S					
T					
R					
I					
D					
E					

Understanding Threats: Threat Modeling



	Service Technician		-->		Measurement Process	
	Mitigations	Vulnerabilities	Mitigations	Vulnerabilities	Mitigations	Vulnerabilities
S	user + pw				SSL Cert	
T			SSL			
R		no audit log				no audit log
I			SSL			
D			unrel. comm.			
E						no internal access control

Understanding Threats: Threat Modeling

Threat Categories in Detail:

Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege

Understanding Threats: Threat Modeling

Spoofing: Can an attacker gain access using a false identity?

Understanding Threats: Threat Modeling

Spoofing: Can an attacker gain access using a false identity?

- Creating a file before a process does
- DNS spoofing
- Phishing emails
- Taking control of an account

Understanding Threats: Threat Modeling

Spoofing: Can an attacker gain access using a false identity?

- Creating a file before a process does
- DNS spoofing
- Phishing emails
- Taking control of an account

Tampering: Can an attacker modify data as it flows through the application?

Understanding Threats: Threat Modeling

Spoofing: Can an attacker gain access using a false identity?

- Creating a file before a process does
- DNS spoofing
- Phishing emails
- Taking control of an account

Tampering: Can an attacker modify data as it flows through the application?

- Modifying files
- Modifying memory
- Redirecting traffic flows
- Log injection attacks

Understanding Threats: Threat Modeling

Repudiation: If an attacker denies doing something, can we prove he did it?

Understanding Threats: Threat Modeling

Repudiation: If an attacker denies doing something, can we prove he did it?

- Repudiating an action by claiming to not having performed an action
- Repudiating an action by claiming to have performed an action
- Putting attacks in the logs to corrupt the audit trail

Understanding Threats: Threat Modeling

Repudiation: If an attacker denies doing something, can we prove he did it?

- Repudiating an action by claiming to not having performed an action
- Repudiating an action by claiming to have performed an action
- Putting attacks in the logs to corrupt the audit trail

Information Disclosure: Can an attacker gain access to private or potentially injurious data?

Understanding Threats: Threat Modeling

Repudiation: If an attacker denies doing something, can we prove he did it?

- Repudiating an action by claiming to not having performed an action
- Repudiating an action by claiming to have performed an action
- Putting attacks in the logs to corrupt the audit trail

Information Disclosure: Can an attacker gain access to private or potentially injurious data?

- Extracting secrets from the error messages
- Taking advantage of missing access control checks
- Reading data on the network
- Learning who is talking to whom

Understanding Threats: Threat Modeling

Denial of Service: Can an attacker crash or reduce the availability of the system?

Understanding Threats: Threat Modeling

Denial of Service: Can an attacker crash or reduce the availability of the system?

- Consuming excessive amounts of memory
- Using a process as an amplifier
- Filling up the storage space
- Consuming large amounts of network resources

Understanding Threats: Threat Modeling

Denial of Service: Can an attacker crash or reduce the availability of the system?

- Consuming excessive amounts of memory
- Using a process as an amplifier
- Filling up the storage space
- Consuming large amounts of network resources

Elevation of Privilege: Can an attacker assume the identity of a privileged user?

Understanding Threats: Threat Modeling

Denial of Service: Can an attacker crash or reduce the availability of the system?

- Consuming excessive amounts of memory
- Using a process as an amplifier
- Filling up the storage space
- Consuming large amounts of network resources

Elevation of Privilege: Can an attacker assume the identity of a privileged user?

- Corrupting a process by gaining access to memory
- Modifying stored data to perform other actions than the user intended
- Bypass buggy authorisation checks

Understanding Threats: Threat Modeling

STRIDE can be too abstract:

- Ask concrete questions, then go back to the abstract threats.
- Just because you have an authentication mechanism doesn't mean you are secure!

Understanding Threats: Threat Modeling

Question catalogue (Michael Howard)

- How do you authenticate this process? (S)
- What access control settings are on the binaries? (TIR)
- Are binaries digitally signed? (STR)
- Do you have Denial-of-Service (DoS) defences in front of the process? (D)
- What identity does this process run as? (E)
- How do you authenticate the entity? (S)
- Is the identity of the external entity propagated through the system? (R)
- How is the data in transit protected from snooping / tweaking? (TI)
- Do you restrict the protocol that will be used for the communication? (TD)
- What is the sensitivity of the data? (I)
- What permissions are on the data? (TI)
- Do you have any cryptographic defences in place? (TI)

3. Proposing Mitigations

- Authentication
- Integrity
- Non-repudiation
- Confidentiality
- Availability
- Authorisation

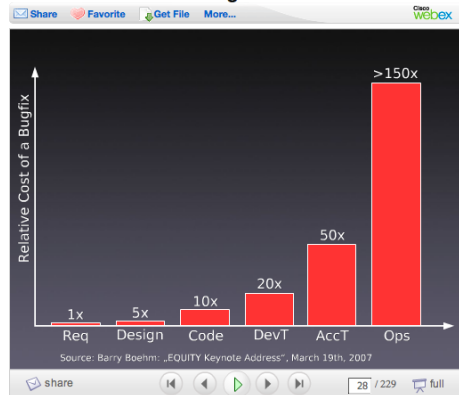
More on mitigations and technology later!

Understanding Threats: Threat Modeling

Why?

- Threat modeling helps you find security problems early in the life-cycle (Requirements or Design vs. Accreditation or Operation)
- Understand and concretise your security requirements
- Complement other security techniques to address more problems

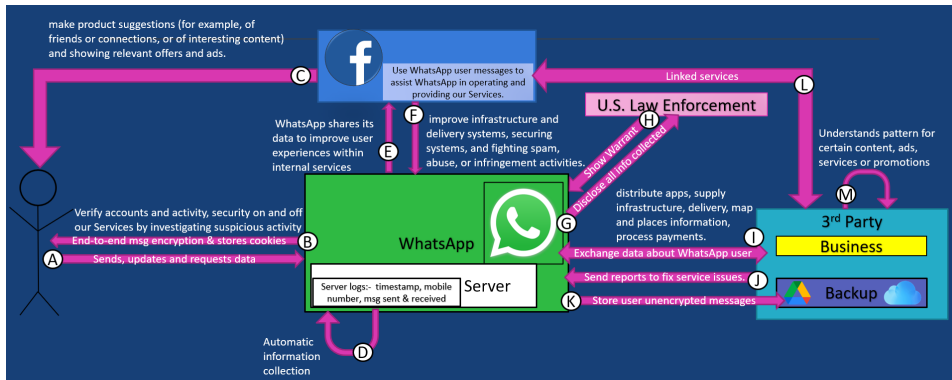
Advanced OOP and Design Patterns



Understanding Threats: Threat Modeling

But...

- You can still use threat modeling at later stages (e.g., accreditation), with different granularity and different objectives
- Example: Privacy threat modeling of WhatsApp based on privacy policy documents [DMK+23] using LINDDUN Go [WSJ20] methodology



References I

- 
- T. Avgerinos, S. K. Cha, A. Rebert, E. J. Schwartz, M. Woo, and D. Brumley.
Automatic exploit generation.
Commun. ACM, 57(2):74–84, 2014.
- 
- M. Busi, J. Noorman, J. Van Bulck, L. Galletta, P. Degano, J. T. Mühlberg, and F. Piessens.
Provably secure isolation for interruptible enclaved execution on small microprocessors.
In *Computer Security Foundations Symposium (CSF)*. IEEE, 2020.
- 
- D. Brumley, P. Poosankam, D. Song, and J. Zheng.
Automatic patch-based exploit generation is possible: Techniques and implications.
In *2008 IEEE Symposium on Security and Privacy (S&P 2008)*, pp. 143–157, 2008.
- 
- C. Cadar, D. Dunbar, D. R. Engler, et al.
Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs.
In *OSDI*, vol. 8, pp. 209–224, 2008.
- 
- C. Canella, J. Van Bulck, M. Schwarz, M. Lipp, B. Von Berg, P. Ortner, F. Piessens, D. Evtushkin, and D. Gruss.
A systematic evaluation of transient execution attacks and defenses.
In *28th USENIX Security Symposium (USENIX Security 19)*, pp. 249–266, 2019.
- 
- H. Depuydt, M. Gülmez, T. Nyman, and J. T. Mühlberg.
Do we still need canaries in the coal mine? measuring shadow stack effectiveness in countering stack smashing.
In *2025 International Conference on Availability, Reliability and Security (ARES)*, vol. 15993 of *LNCS*, pp. 193–205. Springer, 2025.
- 
- Y. Dimova, K. Mrunmayee, S. Kalantari, K. Wuyts, W. Joosen, and J. T. Mühlberg.
From privacy policies to privacy threats: A case study in policy-based threat modeling.
In *Proceedings of the 22nd Workshop on Privacy in the Electronic Society (WPES '23)*, New York, NY, USA, 2023. ACM.

References II



D. Dolev and A. C. Yao.

On the security of public key protocols.

In *Proceedings of the 22Nd Annual Symposium on Foundations of Computer Science, SFCS '81*, pp. 350–357, Washington, DC, USA, 1981. IEEE Computer Society.



P. Eder-Neuhauser, T. Zseby, J. Fabini, and G. Vormayr.

Cyber attack models for smart grid environments.

Sustainable Energy, Grids and Networks, 12:10–29, December 2017.



Ú. Erlingsson, Y. Younan, and F. Piessens.

Low-Level Software Security by Example, pp. 633–658.

Springer Berlin Heidelberg, Berlin, Heidelberg, 2010.



P. Godefroid, M. Y. Levin, and D. Molnar.

Automated whitebox fuzz testing.

In *NDSS '08*. Internet Society (ISOC), 2008.



S. K. Huang, M. H. Huang, P. Y. Huang, H. L. Lu, and C. W. Lai.

Software crash analysis for automatic exploit generation on binary programs.

IEEE Transactions on Reliability, 63(1):270–289, 2014.



C. He, M. Sundararajan, A. Datta, A. Derek, and J. C. Mitchell.

A modular correctness proof of ieee 802.11i and tls.

In *Proceedings of the 12th ACM Conference on Computer and Communications Security, CCS '05*, pp. 2–15, New York, NY, USA, 2005. ACM.

References III

- 
- B. Jacobs, J. Smans, and F. Piessens.
VeriFast: Imperative programs as proofs.
In VSTTE 2010 workshop proceedings, pp. 63–72, 2010.
- 
- P. Kocher, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom.
Spectre attacks: Exploiting speculative execution.
ArXiv e-prints, January 2018.
- 
- P. Maene, J. Götzfried, R. de Clercq, T. Müller, F. Freiling, and I. Verbauwhede.
Hardware-based trusted computing architectures for isolation and attestation.
IEEE Transactions on Computers, PP(99):1–1, 2017.
- 
- J. T. Mühlberg and G. Lüttgen.
Symbolic object code analysis.
In SPIN '10, vol. 6349 of *LNCS*, pp. 4–21, Heidelberg, 2010. Springer.
- 
- C. Miller and C. Valasek.
Remote exploitation of an unaltered passenger vehicle.
Black Hat USA, 2015.
- 
- J. Noorman, J. T. Mühlberg, and F. Piessens.
Authentic execution of distributed event-driven applications with a small TCB.
In STM '17, vol. 10547 of *LNCS*, pp. 55–71, Heidelberg, 2017. Springer.
- 
- J. Noorman, J. Van Bulck, J. T. Mühlberg, F. Piessens, P. Maene, B. Preneel, I. Verbauwhede, J. Götzfried, T. Müller, and F. Freiling.
Sancus 2.0: A low-cost security architecture for IoT devices.
ACM Transactions on Privacy and Security (TOPS), 20:7:1–7:33, 2017.

References IV



D. Papp, Z. Ma, and L. Buttyan.

Embedded systems security: Threats, vulnerabilities, and attack taxonomy.
In 2015 13th Annual Conference on Privacy, Security and Trust (PST), pp. 145–152. IEEE, 2015.



P. Philippaerts, J. T. Mühlberg, W. Penninckx, J. Smans, B. Jacobs, and F. Piessens.

Software verification with VeriFast: Industrial case studies.
Science of Computer Programming (SCP), 82:77–97, 2014.



F. Piessens and I. Verbauwheide.

Software security: Vulnerabilities and countermeasures for two attacker models.
In 2016 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 990–999, 2016.



F. Piessens and P. C. van Oorschot.

Side-channel attacks: A short tour.
IEEE Security & Privacy, 22(2):75–80, 2024.



B. Schneier.

Internet hacking is about to get much worse.
The New York Times, 10 2018.



A. Shostack.

Threat modeling: Designing for security.
John Wiley & Sons, 2014.



M. Schneider, R. J. Masti, S. Shinde, S. Capkun, and R. Perez.

Sok: Hardware-supported trusted execution environments.
arXiv preprint arXiv:2205.12742, 2022.

References V



G. Scopelliti, S. Pouyanrad, J. Noorman, F. Alder, C. Baumann, F. Piessens, and J. T. Mühlberg.
End-to-End Security for Distributed Event-Driven Enclave Applications on Heterogeneous TEEs.
ACM Transactions on Privacy and Security, 2023.



J. Szefer.
Principles of Secure Processor Architecture Design, pp. 113–124.
Springer International Publishing, Cham, 2019.



N. Tillmann and J. de Halleux.
Pex – White Box Test Generation for .NET, pp. 134–153.
Springer Berlin Heidelberg, Berlin, Heidelberg, 2008.



J. Van Bulck, J. T. Mühlberg, and F. Piessens.
VulCAN: Efficient component authentication and software isolation for automotive control networks.
In *ACSAC '17*, pp. 225–237, New York, NY, USA, 2017. ACM.



J. Van Bulck, F. Piessens, and R. Strackx.
Nemesis: Studying microarchitectural timing leaks in rudimentary cpu interrupt logic.
In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pp. 178–195. ACM, 2018.



M. Vanhoef and F. Piessens.
Key reinstallation attacks: Forcing nonce reuse in wpa2.
In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, pp. 1313–1328, New York, NY, USA, 2017. Association for Computing Machinery.

References VI



J. Wilson, M. Asplund, N. Johansson, and F. Boeira.

Provably secure communication protocols for remote attestation.

In *Proceedings of the 19th International Conference on Availability, Reliability and Security, ARES '24*, New York, NY, USA, 2024. Association for Computing Machinery.



H. Winderix, J. T. Mühlberg, and F. Piessens.

Compiler-assisted hardening of embedded software against interrupt latency side-channel attacks.

In *EuroS&P '21*, pp. 667 – 682, Washington, DC, USA, 2021. IEEE.



K. Wuyts, L. Sion, and W. Joosen.

Linddun go: A lightweight approach to privacy threat modeling.

In *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 302–309, USA, 2020. IEEE.



M. Zalewski.

American Fuzzy Lop: A security-oriented fuzzer, 2010.

<http://lcamtuf.coredump.cx/afl/>.